

Missão: Analista de Banco de Dados da Escola Tech

Contexto

É seu primeiro dia como Analista de Banco de Dados da Escola Tech. A secretaria informa que o sistema antigo foi substituído por um banco SQLite e várias solicitações chegaram para você resolver. Cada solicitação representa uma **Ordem de Serviço (OS)**. Você deverá analisar cada uma e executar as operações necessárias.

OS-001 – Preparação do Banco_PARTES_1_2_3

O banco ainda não possui tabelas.

A secretaria solicita que seja criada a estrutura necessária para armazenar os alunos e as disciplinas.

Entregável: Estrutura pronta para receber os dados.

OS-002 – Migração de Dados_PARTE_4

Os dados antigos foram exportados em arquivos CSV.

Importe os arquivos recebidos para o banco de dados.

Após a importação, confirme se todos os registros foram carregados corretamente.

OS-003 – Auditoria_PARTE_8_9

A direção quer verificar se a migração ocorreu corretamente.

Gere consultas que permitam visualizar:

- todos os alunos;
- apenas os nomes;
- notas dos estudantes;
- lista de professores.

OS-004 – Correção de Cadastro _PARTE_10_11

Foram encontrados erros nas informações.

Realize as correções necessárias nos registros informados pela secretaria.

OS-005 – Novas Matrículas_PARTE_12

Cinco novos alunos realizaram matrícula nesta semana.

Cadastre-os no sistema.

OS-006 – Limpeza do Banco

Durante a auditoria foram encontrados:

- registros duplicados;
- alunos desistentes;
- disciplinas canceladas.

Faça a limpeza do banco mantendo apenas os dados válidos.

OS-007 – Nova Funcionalidade

A escola agora deseja registrar a cidade dos alunos e o laboratório utilizado por cada disciplina.

Adapte o banco de dados sem perder os registros existentes.

OS-008 – Relatórios Gerenciais

A direção precisa responder perguntas como:

- Quantos alunos existem?
- Qual é a média geral?
- Quem possui a maior nota?
- Quem possui a menor nota?
- Qual turma possui mais alunos?
- Quais são os melhores estudantes?

Produza os relatórios.

OS-009 – Classificação dos Alunos

A coordenação deseja que o sistema apresente automaticamente a situação de cada aluno.

Os estudantes deverão aparecer como:

- Aprovado
- Recuperação
- Reprovado

OS-010 – Encerramento da Migração

Depois que o banco estiver validado:

- exporte os relatórios em CSV;
- entregue os arquivos para a direção;
- remova as tabelas temporárias utilizadas durante a migração.

SEQUÊNCIA DOS COMANDOS

Parte 5 – Importando o arquivo alunos.csv

No ambiente SQLite Online, procure uma opção semelhante a:

```
Import
Import CSV
File → Import
CSV Import
```

Selecione o arquivo `alunos.csv`.

Durante a importação:

1. escolha a tabela `Alunos`;
2. informe que a primeira linha contém os nomes das colunas;
3. selecione a vírgula como separador;
4. confirme a importação.

Após importar, execute:

```
SELECT *
FROM Alunos;
```

Conte quantos registros foram importados:

```
SELECT COUNT(*) AS total_alunos
FROM Alunos;
```

Parte 7 – Importação utilizando comandos do SQLite

Em algumas ferramentas que utilizam o terminal do SQLite, a importação pode ser feita com comandos próprios do programa.

Esses comandos não são SQL tradicional, mas comandos do terminal SQLite:

```
.mode csv
.import alunos.csv Alunos
```

Para as disciplinas:

```
.mode csv
.import disciplinas.csv Disciplinas
```

Caso o arquivo contenha cabeçalho e a ferramenta importe essa primeira linha como um registro, será necessário excluir o cabeçalho importado incorretamente.

Exemplo:

```
DELETE FROM Alunos
WHERE id = 'id';
```

Esse procedimento pode variar de acordo com o ambiente utilizado. Em editores online, prefira a ferramenta visual de importação.

Parte 8 – Conferindo os dados importados

Visualizar todos os alunos

```
SELECT *  
FROM Alunos;
```

Visualizar apenas os nomes

```
SELECT nome  
FROM Alunos;
```

Visualizar nome, turma e nota

```
SELECT nome, turma, nota  
FROM Alunos;
```

Visualizar todas as disciplinas

```
SELECT *  
FROM Disciplinas;
```

Visualizar disciplina e professor

```
SELECT disciplina, professor  
FROM Disciplinas;
```

Visualizar disciplina e carga horária

```
SELECT disciplina, carga_horaria  
FROM Disciplinas;
```

Parte 9 – Realizando uma auditoria dos dados

Antes de alterar qualquer informação, faça consultas para localizar os registros.

Alunos com nota menor que 7

```
SELECT *  
FROM Alunos  
WHERE nota < 7;
```

Alunos da turma Info A

```
SELECT *  
FROM Alunos  
WHERE turma = 'Info A';
```

Alunos com idade maior que 20 anos

```
SELECT *  
FROM Alunos  
WHERE idade > 20;
```

Disciplinas com carga horária de 80 horas

```
SELECT *  
FROM Disciplinas  
WHERE carga_horaria = 80;
```

Professores que ministram uma disciplina específica

```
SELECT professor, disciplina  
FROM Disciplinas  
WHERE disciplina = 'Banco de Dados';
```

Parte 10 – Corrigindo o cadastro dos alunos

A secretaria informou que alguns dados precisam ser corrigidos.

Alterar a turma de um aluno

Antes da alteração, localize o registro:

```
SELECT *  
FROM Alunos  
WHERE nome = 'Diego Alves';
```

Agora atualize a turma:

```
UPDATE Alunos  
SET turma = 'Info B'  
WHERE nome = 'Diego Alves';
```

Confira a alteração:

```
SELECT *  
FROM Alunos  
WHERE nome = 'Diego Alves';
```

Corrigir a nota de um aluno

```
UPDATE Alunos
SET nota = 7.2
WHERE nome = 'Felipe Melo';
```

Confira:

```
SELECT nome, nota
FROM Alunos
WHERE nome = 'Felipe Melo';
```

Corrigir a nota de outro aluno

```
UPDATE Alunos
SET nota = 9.0
WHERE nome = 'Ana Souza';
```

Atualizar a idade de um aluno

```
UPDATE Alunos
SET idade = 20
WHERE nome = 'Ana Souza';
```

Parte 11 – Corrigindo o cadastro das disciplinas

A escola informou que um professor mudou de disciplina.

Antes de alterar, consulte:

```
SELECT *
FROM Disciplinas
WHERE disciplina = 'Programação Web';
```

Atualize o nome do professor:

```
UPDATE Disciplinas
SET professor = 'Fernanda Oliveira'
WHERE disciplina = 'Programação Web';
```

Confira:

```
SELECT disciplina, professor
FROM Disciplinas
WHERE disciplina = 'Programação Web';
```

O comando `UPDATE` deve ser utilizado com `WHERE`. Sem essa condição, todos os registros da tabela podem ser alterados.

Parte 12 – Cadastrando novos alunos

Cadastre cinco novos alunos.

```
INSERT INTO Alunos (nome, idade, turma, nota)
VALUES ('Igor Martins', 18, 'Info B', 8.1);
INSERT INTO Alunos (nome, idade, turma, nota)
VALUES ('Joana Ferreira', 19, 'Redes A', 7.8);
INSERT INTO Alunos (nome, idade, turma, nota)
VALUES ('Lucas Pereira', 21, 'Administração A', 6.5);
INSERT INTO Alunos (nome, idade, turma, nota)
VALUES ('Mariana Lopes', 20, 'Info A', 9.4);
INSERT INTO Alunos (nome, idade, turma, nota)
VALUES ('Nicolas Gomes', 22, 'Redes A', 4.8);
```

Também é possível inserir vários registros em um único comando:

```
INSERT INTO Alunos (nome, idade, turma, nota)
VALUES
    ('Olivia Ramos', 18, 'Info A', 8.3),
    ('Paulo Mendes', 20, 'Redes A', 7.1),
    ('Renata Lima', 19, 'Administração A', 9.0);
```

Confira os novos registros:

```
SELECT *
FROM Alunos
ORDER BY id DESC;
```

Parte 13 – Cadastrando uma nova disciplina

```
INSERT INTO Disciplinas (
    disciplina,
    professor,
    carga_horaria
)
VALUES (
    'Segurança da Informação',
    'André Barros',
    60
);
```

Confira:

```
SELECT *
FROM Disciplinas;
```

Parte 14 – Excluindo um aluno específico

Antes de excluir, localize o aluno:

```
SELECT *
FROM Alunos
WHERE nome = 'Henrique Silva';
```


Depois execute:

```
DELETE FROM Alunos  
WHERE nome = 'Henrique Silva';
```

Confira se o registro foi removido:

```
SELECT *  
FROM Alunos  
WHERE nome = 'Henrique Silva';
```

Se a consulta não retornar resultados, o aluno foi excluído.

Parte 15 – Excluindo alunos com nota muito baixa

Antes de excluir, verifique quais registros serão afetados:

```
SELECT *  
FROM Alunos  
WHERE nota < 4;
```

Depois execute:

```
DELETE FROM Alunos  
WHERE nota < 4;
```

Confira:

```
SELECT *  
FROM Alunos  
ORDER BY nota ASC;
```

É uma boa prática executar primeiro um `SELECT` com a mesma condição que será usada no `DELETE`.

Parte 16 – Excluindo uma disciplina cancelada

Localize a disciplina:

```
SELECT *  
FROM Disciplinas  
WHERE disciplina = 'Engenharia de Software';
```

Exclua:

```
DELETE FROM Disciplinas  
WHERE disciplina = 'Engenharia de Software';
```

Confira:

```
SELECT *  
FROM Disciplinas;
```

Parte 17 – Alterando a estrutura da tabela Alunos

A escola deseja registrar a cidade de cada aluno.

Adicione a coluna:

```
ALTER TABLE Alunos  
ADD COLUMN cidade TEXT;
```

Confira a nova estrutura:

```
PRAGMA table_info(Alunos);
```

A coluna `cidade` deverá aparecer no resultado.

Parte 18 – Alterando a estrutura da tabela Disciplinas

A escola também deseja registrar o laboratório utilizado em cada disciplina.

Execute:

```
ALTER TABLE Disciplinas  
ADD COLUMN laboratorio TEXT;
```

Confira:

```
PRAGMA table_info(Disciplinas);
```

Parte 19 – Atualizando as cidades dos alunos

```
UPDATE Alunos  
SET cidade = 'Recife'  
WHERE nome = 'Ana Souza';  
UPDATE Alunos  
SET cidade = 'Olinda'  
WHERE nome = 'Bruno Lima';  
UPDATE Alunos  
SET cidade = 'Jaboatão dos Guararapes'  
WHERE nome = 'Carla Santos';  
UPDATE Alunos  
SET cidade = 'Paulista'  
WHERE nome = 'Diego Alves';  
UPDATE Alunos  
SET cidade = 'Recife'  
WHERE nome = 'Elisa Rocha';
```

Consulte:

```
SELECT nome, cidade
FROM Alunos
WHERE cidade IS NOT NULL;
```

Parte 20 – Atualizando os laboratórios

```
UPDATE Disciplinas
SET laboratorio = 'Laboratório 1'
WHERE disciplina = 'Banco de Dados';
UPDATE Disciplinas
SET laboratorio = 'Laboratório 2'
WHERE disciplina = 'Programação Web';
UPDATE Disciplinas
SET laboratorio = 'Laboratório de Redes'
WHERE disciplina = 'Redes de Computadores';
UPDATE Disciplinas
SET laboratorio = 'Laboratório 1'
WHERE disciplina = 'Lógica de Programação';
```

Confira:

```
SELECT disciplina, laboratorio
FROM Disciplinas;
```

Parte 21 – Contando os alunos

```
SELECT COUNT(*) AS total_alunos
FROM Alunos;
```

O COUNT (*) conta todos os registros da tabela.

Parte 22 – Calculando a média das notas

```
SELECT AVG(nota) AS media_geral
FROM Alunos;
```

Para apresentar a média com duas casas decimais:

```
SELECT ROUND(AVG(nota), 2) AS media_geral
FROM Alunos;
```

Parte 23 – Encontrando a maior nota

```
SELECT MAX(nota) AS maior_nota
FROM Alunos;
```

Para descobrir quem possui a maior nota:

```
SELECT nome, nota
FROM Alunos
```

```
WHERE nota = (  
    SELECT MAX(nota)  
    FROM Alunos  
);
```

Parte 24 – Encontrando a menor nota

```
SELECT MIN(nota) AS menor_nota  
FROM Alunos;
```

Para descobrir quem possui a menor nota:

```
SELECT nome, nota  
FROM Alunos  
WHERE nota = (  
    SELECT MIN(nota)  
    FROM Alunos  
);
```

Parte 25 – Ordenando os alunos

Ordem alfabética

```
SELECT *  
FROM Alunos  
ORDER BY nome ASC;
```

Da maior para a menor nota

```
SELECT nome, turma, nota  
FROM Alunos  
ORDER BY nota DESC;
```

Da menor para a maior nota

```
SELECT nome, turma, nota  
FROM Alunos  
ORDER BY nota ASC;
```

Por turma e depois por nome

```
SELECT nome, turma, nota  
FROM Alunos  
ORDER BY turma ASC, nome ASC;
```

Parte 26 – Contando alunos por turma

```
SELECT  
    turma,  
    COUNT(*) AS quantidade_alunos  
FROM Alunos
```

```
GROUP BY turma;
```

Para listar da turma com mais alunos para a turma com menos alunos:

```
SELECT
    turma,
    COUNT(*) AS quantidade_alunos
FROM Alunos
GROUP BY turma
ORDER BY quantidade_alunos DESC;
```

Parte 27 – Calculando a média por turma

```
SELECT
    turma,
    ROUND(AVG(nota), 2) AS media_turma
FROM Alunos
GROUP BY turma
ORDER BY media_turma DESC;
```

Essa consulta mostra o desempenho médio de cada turma.

Parte 28 – Classificando os alunos com CASE WHEN

Crie uma consulta com a situação de cada estudante.

```
SELECT
    nome,
    turma,
    nota,
    CASE
        WHEN nota >= 7 THEN 'Aprovado'
        WHEN nota >= 5 THEN 'Recuperação'
        ELSE 'Reprovado'
    END AS situacao
FROM Alunos;
```

Explicação

- notas iguais ou maiores que 7: Aprovado;
- notas entre 5 e 6,9: Recuperação;
- notas menores que 5: Reprovado.

Parte 29 – Ordenando o relatório de situação

```
SELECT
    nome,
    turma,
    nota,
    CASE
        WHEN nota >= 7 THEN 'Aprovado'
```

```

        WHEN nota >= 5 THEN 'Recuperação'
        ELSE 'Reprovado'
    END AS situacao
FROM Alunos
ORDER BY nota DESC;

```

Parte 30 – Contando alunos por situação

```

SELECT
    CASE
        WHEN nota >= 7 THEN 'Aprovado'
        WHEN nota >= 5 THEN 'Recuperação'
        ELSE 'Reprovado'
    END AS situacao,
    COUNT(*) AS quantidade
FROM Alunos
GROUP BY
    CASE
        WHEN nota >= 7 THEN 'Aprovado'
        WHEN nota >= 5 THEN 'Recuperação'
        ELSE 'Reprovado'
    END;

```

Parte 31 – Criando um relatório completo dos alunos

Execute:

```

SELECT
    id,
    nome,
    idade,
    turma,
    nota,
    cidade,
    CASE
        WHEN nota >= 7 THEN 'Aprovado'
        WHEN nota >= 5 THEN 'Recuperação'
        ELSE 'Reprovado'
    END AS situacao
FROM Alunos
ORDER BY turma, nome;

```

Esse relatório poderá ser exportado para CSV.

Parte 32 – Criando o relatório dos aprovados

```

SELECT
    id,
    nome,
    idade,
    turma,
    nota,
    cidade
FROM Alunos

```

```
WHERE nota >= 7
ORDER BY nota DESC;
```

Parte 33 – Criando o relatório de recuperação

```
SELECT
    id,
    nome,
    turma,
    nota
FROM Alunos
WHERE nota >= 5
    AND nota < 7
ORDER BY nota DESC;
```

Parte 34 – Criando o relatório dos reprovados

```
SELECT
    id,
    nome,
    turma,
    nota
FROM Alunos
WHERE nota < 5
ORDER BY nota ASC;
```

Parte 35 – Criando o relatório das disciplinas

```
SELECT
    id,
    disciplina,
    professor,
    carga_horaria,
    laboratorio
FROM Disciplinas
ORDER BY disciplina ASC;
```

Parte 36 – Exportando o relatório completo para CSV

Primeiro, execute a consulta:

```
SELECT
    id,
    nome,
    idade,
    turma,
    nota,
    cidade,
    CASE
        WHEN nota >= 7 THEN 'Aprovado'
        WHEN nota >= 5 THEN 'Recuperação'
        ELSE 'Reprovado'
    END AS situacao
FROM Alunos
ORDER BY turma, nome;
```

No ambiente SQLite Online, procure uma opção semelhante a:

```
Export  
Download CSV  
Save as CSV  
Export Results
```

Salve o arquivo com o nome:

```
relatorio_alunos.csv
```

Parte 37 – Exportando os aprovados

Execute:

```
SELECT  
    id,  
    nome,  
    idade,  
    turma,  
    nota,  
    cidade  
FROM Alunos  
WHERE nota >= 7  
ORDER BY nota DESC;
```

Exporte o resultado com o nome:

```
aprovados.csv
```

Parte 38 – Exportando as disciplinas

Execute:

```
SELECT  
    id,  
    disciplina,  
    professor,  
    carga_horaria,  
    laboratorio  
FROM Disciplinas  
ORDER BY disciplina;
```

Exporte com o nome:

```
disciplinas_atualizadas.csv
```


Parte 39 – Exportação pelo terminal SQLite

Em ambientes que permitem os comandos do terminal SQLite, a exportação pode ser feita da seguinte forma:

```
.headers on
.mode csv
.output relatorio_alunos.csv

SELECT
    id,
    nome,
    idade,
    turma,
    nota,
    cidade,
    CASE
        WHEN nota >= 7 THEN 'Aprovado'
        WHEN nota >= 5 THEN 'Recuperação'
        ELSE 'Reprovado'
    END AS situacao
FROM Alunos
ORDER BY turma, nome;

.output stdout
```

Para exportar os aprovados:

```
.headers on
.mode csv
.output aprovados.csv

SELECT
    id,
    nome,
    idade,
    turma,
    nota,
    cidade
FROM Alunos
WHERE nota >= 7
ORDER BY nota DESC;

.output stdout
```

Os comandos `.headers`, `.mode` e `.output` pertencem ao programa de terminal do SQLite. Eles podem não funcionar em todos os editores online.

Parte 40 – Criando uma tabela temporária de importação

Em uma situação real, pode ser útil importar os dados primeiro para uma tabela temporária.

```
CREATE TABLE Alunos_Importacao (
```

```
id INTEGER,  
nome TEXT,  
idade INTEGER,  
turma TEXT,  
nota REAL  
);
```

Depois de importar o CSV para essa tabela, confira:

```
SELECT *  
FROM Alunos_Importacao;
```

Copie os dados válidos para a tabela principal:

```
INSERT INTO Alunos (id, nome, idade, turma, nota)  
SELECT id, nome, idade, turma, nota  
FROM Alunos_Importacao  
WHERE nome IS NOT NULL  
AND nota BETWEEN 0 AND 10;
```

Essa estratégia ajuda a validar os dados antes de colocá-los na tabela definitiva.

Parte 41 – Removendo a tabela temporária

Depois que os dados forem conferidos e transferidos, execute:

```
DROP TABLE Alunos_Importacao;
```

Confira se ela foi removida:

```
SELECT name  
FROM sqlite_master  
WHERE type = 'table';
```

Parte 42 – Removendo a tabela Disciplinas

Antes de remover a tabela, confira os dados:

```
SELECT *  
FROM Disciplinas;
```

Depois execute:

```
DROP TABLE Disciplinas;
```

Tente consultar novamente:

```
SELECT *  
FROM Disciplinas;
```

O SQLite deverá apresentar uma mensagem semelhante a:

```
no such table: Disciplinas
```

Isso significa que a tabela não existe mais.

Parte 43 – Diferença entre DELETE e DROP TABLE

DELETE

Remove registros, mas mantém a tabela.

```
DELETE FROM Alunos  
WHERE nome = 'Ana Souza';
```

A estrutura da tabela continua existindo.

DROP TABLE

Remove a tabela completa, incluindo estrutura e registros.

```
DROP TABLE Alunos;
```

Depois desse comando, a tabela deixa de existir.

Parte 44 – Salvando uma cópia de segurança

Antes de executar comandos como `DELETE` ou `DROP TABLE`, faça uma cópia do banco.

Dependendo do ambiente, utilize uma opção como:

```
Save Database  
Export Database  
Download SQLite  
Download DB
```

Salve uma cópia com o nome:

```
EscolaTech_backup.db
```

Essa cópia poderá ser utilizada para restaurar os dados em caso de erro.

Parte 45 – Script completo de criação

```
CREATE TABLE Alunos (  
    id INTEGER PRIMARY KEY,  
    nome TEXT NOT NULL,  
    idade INTEGER,  
    turma TEXT,  
    nota REAL  
);  
  
CREATE TABLE Disciplinas (  
    id INTEGER PRIMARY KEY,  
    disciplina TEXT NOT NULL,  
    professor TEXT,  
    carga_horaria INTEGER  
);
```

Parte 46 – Script completo das alterações estruturais

```
ALTER TABLE Alunos  
ADD COLUMN cidade TEXT;  
  
ALTER TABLE Disciplinas  
ADD COLUMN laboratorio TEXT;
```

Parte 47 – Script do relatório gerencial

```
SELECT COUNT(*) AS total_alunos  
FROM Alunos;  
  
SELECT ROUND(AVG(nota), 2) AS media_geral  
FROM Alunos;  
  
SELECT MAX(nota) AS maior_nota  
FROM Alunos;  
  
SELECT MIN(nota) AS menor_nota  
FROM Alunos;  
  
SELECT  
    turma,  
    COUNT(*) AS quantidade_alunos,  
    ROUND(AVG(nota), 2) AS media_turma  
FROM Alunos  
GROUP BY turma  
ORDER BY quantidade_alunos DESC;
```

Parte 48 – Script do relatório de desempenho

```
SELECT  
    nome,  
    turma,  
    nota,  
    CASE  
        WHEN nota >= 7 THEN 'Aprovado'  
        WHEN nota >= 5 THEN 'Recuperação'  
        ELSE 'Reprovado'  
    END AS situacao
```

```
FROM Alunos  
ORDER BY nota DESC;
```

Parte 49 – Checklist da missão

Antes de entregar a atividade, verifique se você realizou todos os itens.

- ☐ Criou o banco de dados;
- ☐ criou a tabela `Alunos`;
- ☐ criou a tabela `Disciplinas`;
- ☐ importou `alunos.csv`;
- ☐ importou `disciplinas.csv`;
- ☐ verificou a quantidade de registros;
- ☐ realizou consultas com `SELECT`;
- ☐ utilizou `WHERE`;
- ☐ corrigiu registros com `UPDATE`;
- ☐ cadastrou novos registros com `INSERT`;
- ☐ removeu registros com `DELETE`;
- ☐ adicionou novas colunas com `ALTER TABLE`;
- ☐ utilizou `COUNT`;
- ☐ utilizou `AVG`;
- ☐ utilizou `MAX`;
- ☐ utilizou `MIN`;
- ☐ utilizou `ORDER BY`;
- ☐ utilizou `GROUP BY`;
- ☐ utilizou `CASE WHEN`;
- ☐ exportou relatórios em CSV;
- ☐ criou uma cópia de segurança;
- ☐ removeu uma tabela com `DROP TABLE`.

Parte 50 – Entregáveis

O aluno deverá entregar:

1. o arquivo do banco de dados:

EscolaTech.db

2. o arquivo com os comandos executados:

missao_escola_tech.sql

3. os arquivos importados:

alunos.csv
disciplinas.csv

4. os relatórios exportados:

relatorio_alunos.csv
aprovados.csv
disciplinas_atualizadas.csv

5. um relatório curto explicando:

- quais problemas foram encontrados;
- quais correções foram realizadas;
- quantos alunos foram importados;
- quantos alunos foram aprovados;
- qual foi a média geral;
- qual turma possui mais alunos;
- quais cuidados devem ser tomados ao usar UPDATE, DELETE e DROP TABLE.